CMIS 3150: Game Programming & Development I
CSC4820/6820: Interactive Computer Graphics

Instructor: Ying Zhu

# Project 1

In this project, you will develop a simple 3D game. You will learn how to implement player control, collision detection, and collectible objects.

**Due date: 11:59 pm, 10/6/2026**

**Requirements**:

1. This is an individual project. Every student must submit his/her own work.

2. You must use Unity and C#. The project must involve programming. Do not use Unity's visual scripting tool.

3. You may use AI tools to help you write the code for this project, but you are responsible for every line you submit. That means if AI makes an error, it is your error. You must test your game thoroughly and be prepared to explain how your code works. This applies to code you or an AI wrote.

4. Main character
    a. In this game, the main character will move around the game world, collecting coins and avoiding certain obstacles.
    b. You can use either first-person or third-person control.
    c. The main character should be able to walk (WASD keys or arrow keys), run (shift key), and jump (space key).
    d. The main character should have three lives.

5. Gameworld
    a. The game world must have at least **8** gold-colored coins. You decide where to place the coins. Some coins should hang in mid-air, so the main character needs to jump to touch them.
        i. Make the coins rotate on their own.

    b. There should be at least **6** obstacles that the main character must avoid.
        i. All the obstacles should be red color so they can be easily identified.
        ii. The obstacles can be as simple as boxes and spheres.
        iii. Some of the obstacles should be placed so that the main character must jump to avoid them.
        iv. At least one of the obstacles must be moving or rotating so that the main character must time the jump carefully to avoid the obstacle.

v. You are free to create other objects in the game world. For example, the game world can be a house, a 3D maze, or a city block.

c. Level design. You only need to create one level. Experiment with the placement of the obstacles to adjust the difficulty. Some coins should be easy to collect, but some should be more difficult. For example, make the main character jump over gaps or walk over narrow bridges. The game should be of a medium-level difficulty, not too hard but not too easy.

6. Gameplay
    a. The goal of the game is to collect all the coins and avoid touching the obstacles.
        i. Each time a coin is collected, the coin should disappear. The number of coins collected should be displayed in the console window. When all the coins are collected, display the message “You Win!” in the console window and exit the game.

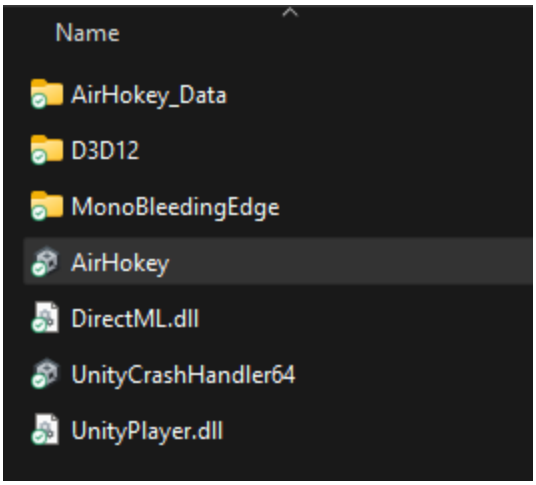
    b. If the main character touches an obstacle, subtract one from the number of lives. When there are 0 lives left, display “Game Over!” in the console window and exit the game.

7. Sound is not required.

8. User Interface: You do not need to create a Graphical User Interface in this project. You can display the messages in the console window as text. Note that Unity’s Console window is only available in the Unity Editor. In a standalone build, these messages are written to the player log file rather than shown on screen. Therefore, you can display these messages as simple on-screen text, but it is not required.

9. Lighting: You can add multiple point lights or a directional light (sunlight) in the Unity Editor. Adding regular lights doesn’t require coding, so this is not very difficult. Experiment with multiple lights.

10. You must build a standalone player of your game for Windows so I can play it without having to open your project in Unity. If you use a Mac, you can create a Mac version. But Windows is preferred.
    a. For instructions on how to create standalone players, visit https://docs.unity3d.com/Manual/BuildSettings.html .
    b. You should test your standalone player before submitting it. Make sure you include all the necessary files, including data files. Do not just submit a single executable file. If I cannot run your game, I may deduct points. A complete standalone player folder on Windows might look like the picture below.




11. Create a short gameplay video with at least one successful playthrough and one failure. The video should be 3 to 5 minutes. You may record your voices to explain your design and gameplay.
    a. You can easily record gameplay on Windows 11 (see this tutorial) and Mac (see this page). I use OBS Studio to record my lectures, and I find it easy to use and reliable.
    b. You may submit the video file to iCollege or upload the video to YouTube (or another cloud drive) and submit a link.

12. Create a separate code walkthrough video. The goal of this video is to demonstrate that you understand how your code controls the behavior of the game. Record your screen with the Unity Editor open and your own project loaded and explain your code in your own voice. The video should be around 5 minutes.
    a. Open each script you wrote and briefly explain what it does. For each script, point at the Inspector and explain which game object the script is attached to.
    b. Walk through one complete interaction from beginning to end, showing what happens in the code. The goal is to demonstrate that you understand the sequence of actions in the code that creates the behavior we see in the game. For example, when the player touches a coin, which callback method is called by the collision, then which method is called to do what, and then which method is called after that. Point at the code on the screen while you explain.
    c. If you are not sure which interaction to choose, use coin collection. It touches input, collision detection, the coin counter, and the win condition, so it shows the most of your code.
    d. You must narrate the video in your own voice. A video with no narration, or one that only reads the code aloud without explaining it, will receive no credit.
    e. **You may use AI tools to help you write the code for this project, but if you cannot explain how your own code works, the project will be graded as incomplete, even if the game runs correctly**.
    f. Submit this video to iCollege or upload it to YouTube (or another cloud drive) and submit a link.

13. Grading guideline
    a. Game assets (max 40 points). Game assets mean the entire Unity project of this game. If you only submit the assets but not a standalone player or gameplay video, I assume your game is unfinished. You will get at most 40 points, depending on how much work you have done. If the game is unfinished and unplayable, you will need to write a report detailing what you have implemented.
    b. Standalone player (30 points). Without a playable standalone player, it's difficult for me to check what you have implemented.
    c. Gameplay demo video (10 points)
    d. Code walkthrough video (20 points)

Game assets + Standalone player + Gameplay video + Code walkthrough video = 100 points

Assuming you submit everything and the standalone player works, the individual game features are checked as below.
    a. Player control
    b. Gameworld
    c. Can the main character collect coins?
    d. Can the main character avoid obstacles?
    e. The game's level of difficulty. Is the game too easy or too difficult to play?
    f. UI (displaying messages in the console window)
    g. Lighting

14. Deliverables. Your final submission should be a zip file that includes the following:
    a. All the game assets and the source code
    b. A standalone application. Include all the files needed to play the standalone application. Do not just include the executable file.
    c. Gameplay video or a file that shows the link to your gameplay video.
    d. Code walkthrough video, or a file that shows the link to your code walkthrough video.
    e. Upload the zip file to iCollege, Assignments, under the folder "Project 1."